Royaume du Maroc
Université Sultan Moulay Slimane
École Nationale des Sciences Appliquées
Béni Mellal

Rapport de Travaux Pratiques

TP 03 Docker — Application Multi-Conteneurs

Module : Génie Logiciel

Filière : « Transformation digitale industrielle **»**

Niveau : 2ème Cycle d'Ingénieur

Année Universitaire : 2025-2026

Réalisé par :

CHAHBI Salma

Demandé par :

Pr. Bahaa Eddine ELBAGHAZAOUI

Introduction

Dans le cadre du module de Génie Logiciel, ce TP a pour objectif de mettre en œuvre une application multi-conteneurs en utilisant Docker. Il permet de comprendre les principes de conteneurisation, l'orchestration des services avec Docker Compose ainsi que la gestion des données persistantes.

Objectifs du TP

Les objectifs principaux sont :

- ✓ Dockeriser une application web simple
- ✓ Utiliser un serveur Nginx pour le frontend
- ✓ Mettre en place une base de données PostgreSQL
- ✓ Configurer pgAdmin pour la gestion de la base
- ✓ Orchestrer les services avec Docker Compose
- ✓ Mettre en place un volume persistant pour conserver les données

Environnement de travail

- Système : Windows
- Outils utilisés :
    - Docker Desktop
    - Docker Compose
    - Navigateur web
    - pgAdmin

Étapes de réalisation

➢ Étape 1 - Préparer l'application

<u>Structure de répertoires :</u>

```
project/
├── frontend/
│   ├── Dockerfile
│   ├── index.html
│   └── .dockerignore
├── backend/
│   └── db.env
└── docker-compose.yml
```

Contenu de frontend/index.html :

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Dockerized App</title>
</head>
<body>
<h1>Bienvenue dans l'application Dockerisée</h1>
</body>
</html>
```

Explication

Le dossier frontend/ contient tout ce qui concerne le serveur web Nginx. Le dossier backend/ centralise les variables d'environnement de la base de données. Le fichier docker compose.yml à la racine orchestre l'ensemble des services.

➢ Étape 2 - Créer le Dockerfile (frontend)

frontend/Dockerfile :

```dockerfile
FROM nginx:alpine

# Copier la page HTML dans le dossier servi par Nginx
COPY index.html /usr/share/nginx/html/index.html

# Exposer le port 80
EXPOSE 80

# Lancer Nginx en foreground
CMD ["nginx", "-g", "daemon off;"]
```

frontend/.dockerignore :

```
*.log
node_modules
```

Le fichier .dockerignore fonctionne comme .gitignore : il exclut les fichiers inutiles du contexte de build pour accélérer la construction.

➢ Étape 3 - Configurer la base de données

backend/db.env :

```
POSTGRES_USER=admin
POSTGRES_PASSWORD=adminpassword
POSTGRES_DB=myapp
```

Les variables de configuration de PostgreSQL sont utilisées pour initialiser la base de données au démarrage du conteneur. La variable **POSTGRES_USER** définit le nom de l'utilisateur, ici « admin », qui sera utilisé pour se connecter à la base de données. La variable **POSTGRES_PASSWORD** correspond au mot de passe associé à cet utilisateur, dans ce cas « adminpassword », et elle est obligatoire pour garantir la sécurité et permettre le démarrage du service. Enfin, la variable **POSTGRES_DB** indique le nom de la base de données qui sera automatiquement créée lors du lancement, ici « myapp ». Ensemble, ces variables permettent de configurer rapidement un environnement PostgreSQL fonctionnel.

➢ Étape 4 — Créer le fichier docker-compose.yml

docker-compose.yml complet :

```yaml
version: "3.8"

services:
  frontend:
    build: ./frontend
    ports:
      - "8080:80"
    networks:
      - app-network
```

```yaml
db:
  image: postgres:13
  environment:
    - POSTGRES_USER=${POSTGRES_USER}
    - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
    - POSTGRES_DB=${POSTGRES_DB}
  env_file:
    - ./backend/db.env
  volumes:
    - db-data:/var/lib/postgresql/data
  networks:
    - app-network

pgadmin:
  image: dpage/pgadmin4
  environment:
    - PGADMIN_DEFAULT_EMAIL=admin@admin.com
    - PGADMIN_DEFAULT_PASSWORD=admin
  ports:
    - "5050:80"
  depends_on:
    - db
  networks:
    - app-network

volumes:
  db-data:

networks:
  app-network:
```

Concepts clés :

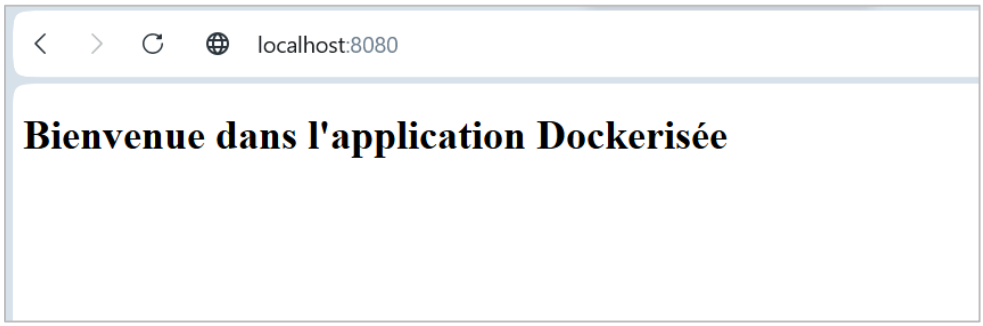
Clé	Explication
build: context	Construit l'image depuis le Dockerfile local au lieu d'en télécharger une
ports	Mappe port_hôte:port_conteneur (ex: 8080:80 = accès sur localhost:8080)
env_file	Charge les variables d'environnement depuis un fichier externe
volumes	Persiste les données PostgreSQL même si le conteneur est supprimé
depends_on	pgAdmin attend que le service db soit démarré avant de se lancer
networks	Tous les services partagent app-network et se voient par leur nom de service

➢ Étape 5 — Tester et valider

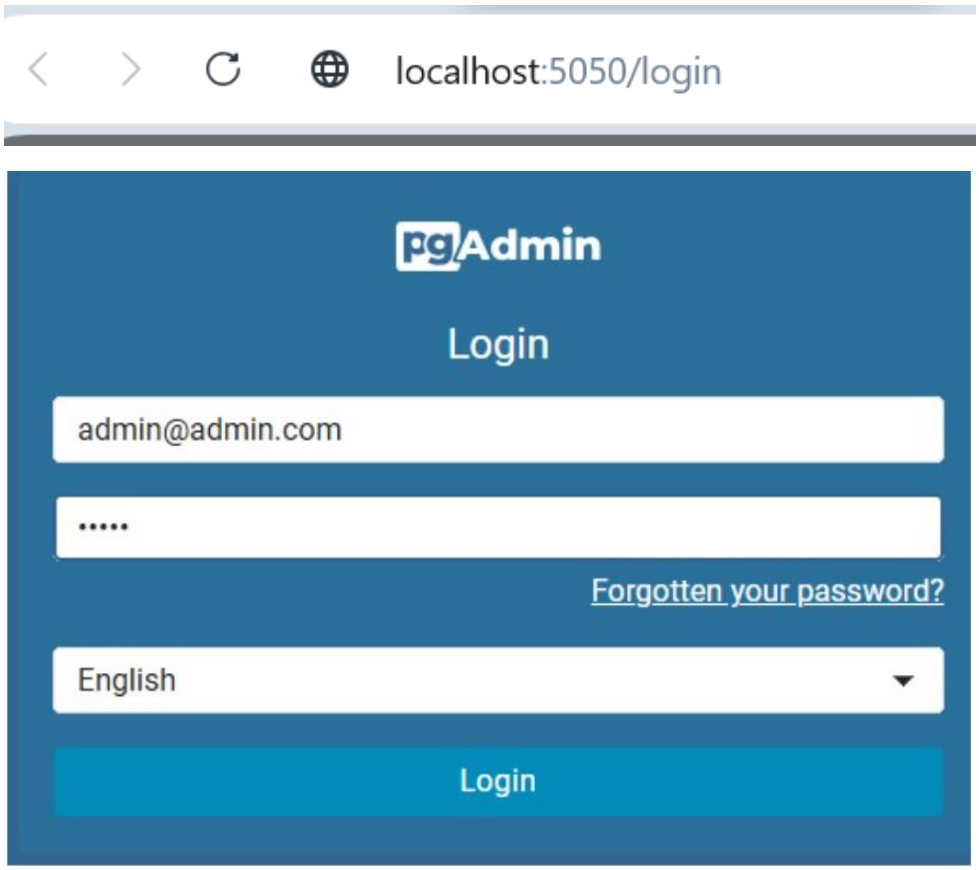
Construire et démarrer les conteneurs

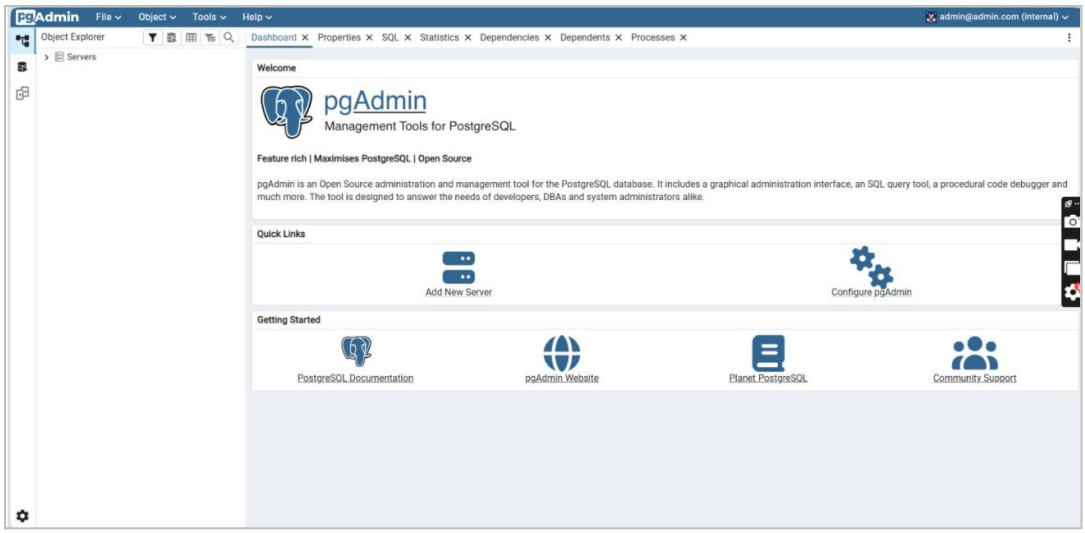
```
C:\Users\DELL\Documents\ENSA\2eme annee cycle\S4\Genie logiciel\TPs\Project>docker-compose up --build
 potential confusion
#1 [internal] load local bake definitions
#1 reading from stdin 649B 0.1s done
#1 DONE 0.1s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 151B 0.0s done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/nginx:alpine
#3 DONE 1.6s

#4 [internal] load .dockerignore
#4 transferring context: 47B 0.0s done
#4 DONE 0.1s

#5 [internal] load build context
db-1        |     You may also use "POSTGRES_HOST_AUTH_METHOD=trust" to allow all
db-1        |     connections without a password. This is *not* recommended.
db-1        |
db-1        |     See PostgreSQL documentation about "trust":
db-1        |     https://www.postgresql.org/docs/current/auth-trust.html
frontend-1  | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
frontend-1  | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
frontend-1  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
frontend-1  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
frontend-1  | /docker-entrypoint.sh: Configuration complete; ready for start up
frontend-1  | 2026/04/23 20:34:49 [notice] 1#1: using the "epoll" event method
frontend-1  | 2026/04/23 20:34:49 [notice] 1#1: nginx/1.29.8
frontend-1  | 2026/04/23 20:34:49 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
frontend-1  | 2026/04/23 20:34:49 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
frontend-1  | 2026/04/23 20:34:49 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
frontend-1  | 2026/04/23 20:34:49 [notice] 1#1: start worker processes
frontend-1  | 2026/04/23 20:34:49 [notice] 1#1: start worker process 30
frontend-1  | 2026/04/23 20:34:49 [notice] 1#1: start worker process 31
frontend-1  | 2026/04/23 20:34:49 [notice] 1#1: start worker process 32
frontend-1  | 2026/04/23 20:34:49 [notice] 1#1: start worker process 33
db-1 exited with code 1
pgadmin-1   | postfix/postlog: starting the Postfix mail system
pgadmin-1   | [2026-04-23 20:35:12 +0000] [1] [INFO] Starting gunicorn 23.0.0
pgadmin-1   | [2026-04-23 20:35:12 +0000] [1] [INFO] Listening at: http://[::]:80 (1)
pgadmin-1   | [2026-04-23 20:35:12 +0000] [1] [INFO] Using worker: gthread
pgadmin-1   | [2026-04-23 20:35:12 +0000] [83] [INFO] Booting worker with pid: 83
pgadmin-1   | /venv/lib/python3.14/site-packages/sshtunnel.py:1040: SyntaxWarning: 'return' in a 'finally' block
pgadmin-1   |   return (ssh_host,
```

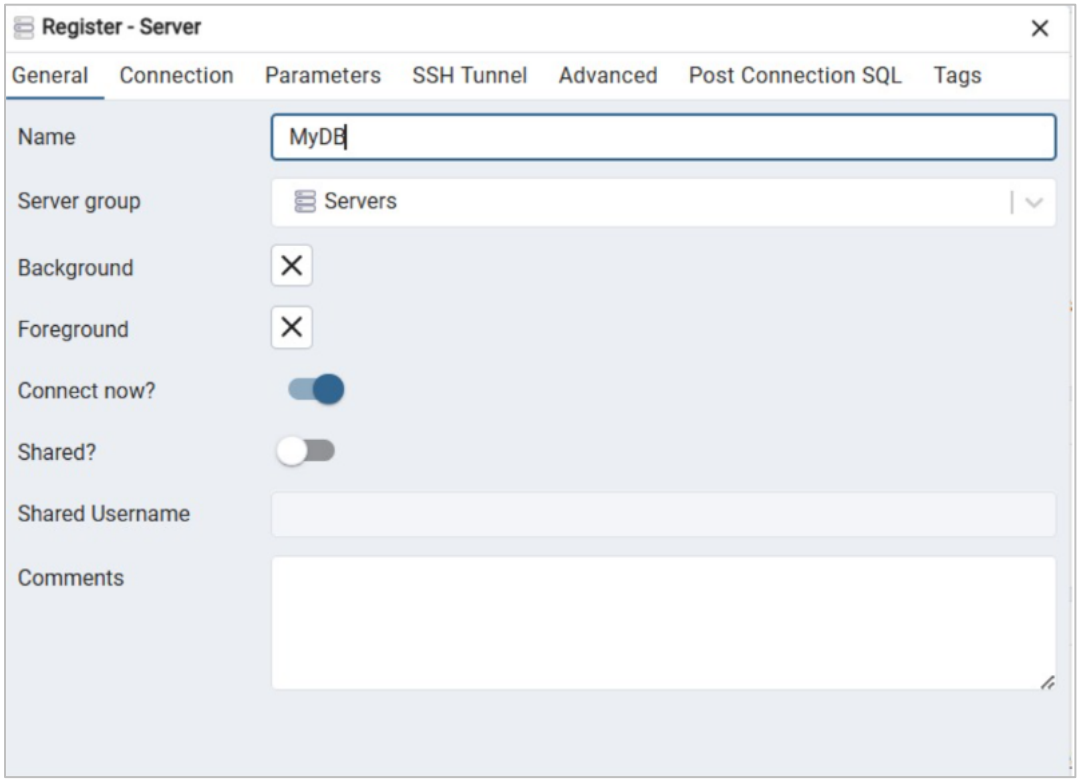
Vérifier le frontend Nginx

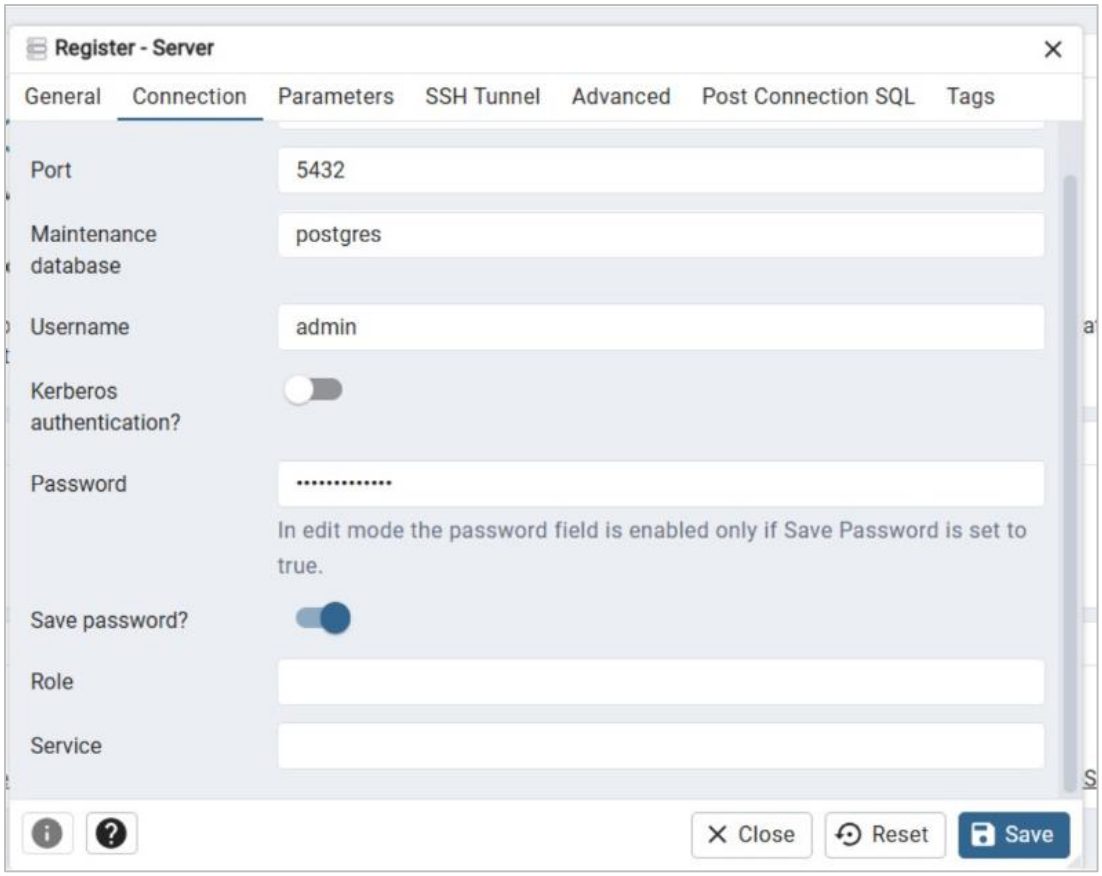


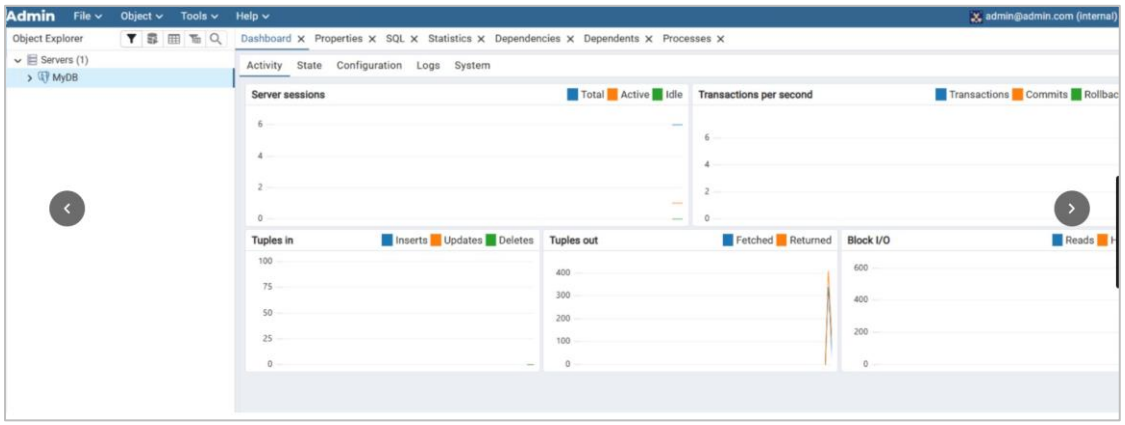
Accéder à pgAdmin



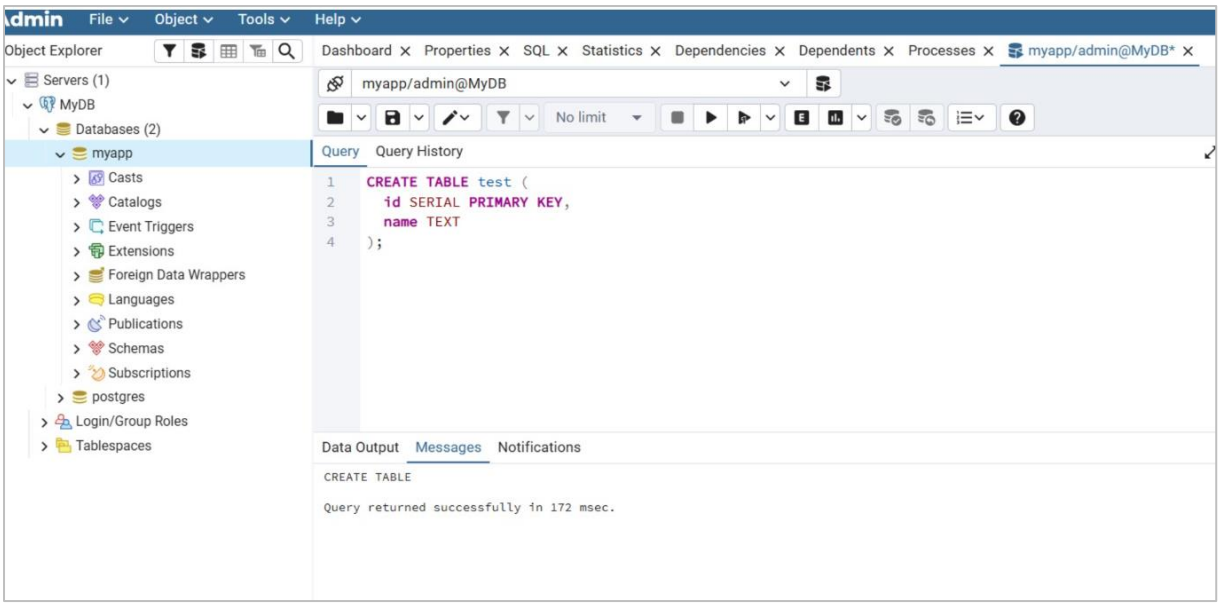

pgAdmin
File
Object
Tools
Help
admin@admin.com (internal)
Object Explorer
Servers
Dashboard
Properties
SQL
Statistics
Dependencies
Dependents
Processes
Welcome
pgAdmin
Management Tools for PostgreSQL
Feature rich | Maximises PostgreSQL | Open Source
pgAdmin is an Open Source administration and management tool for the PostgreSQL database. It includes a graphical administration interface, an SQL query tool, a procedural code debugger and much more. The tool is designed to answer the needs of developers, DBAs and system administrators alike.
Quick Links
Add New Server
Configure pgAdmin
Getting Started
PostgreSQL Documentation
pgAdmin Website
Planet PostgreSQL
Community Support

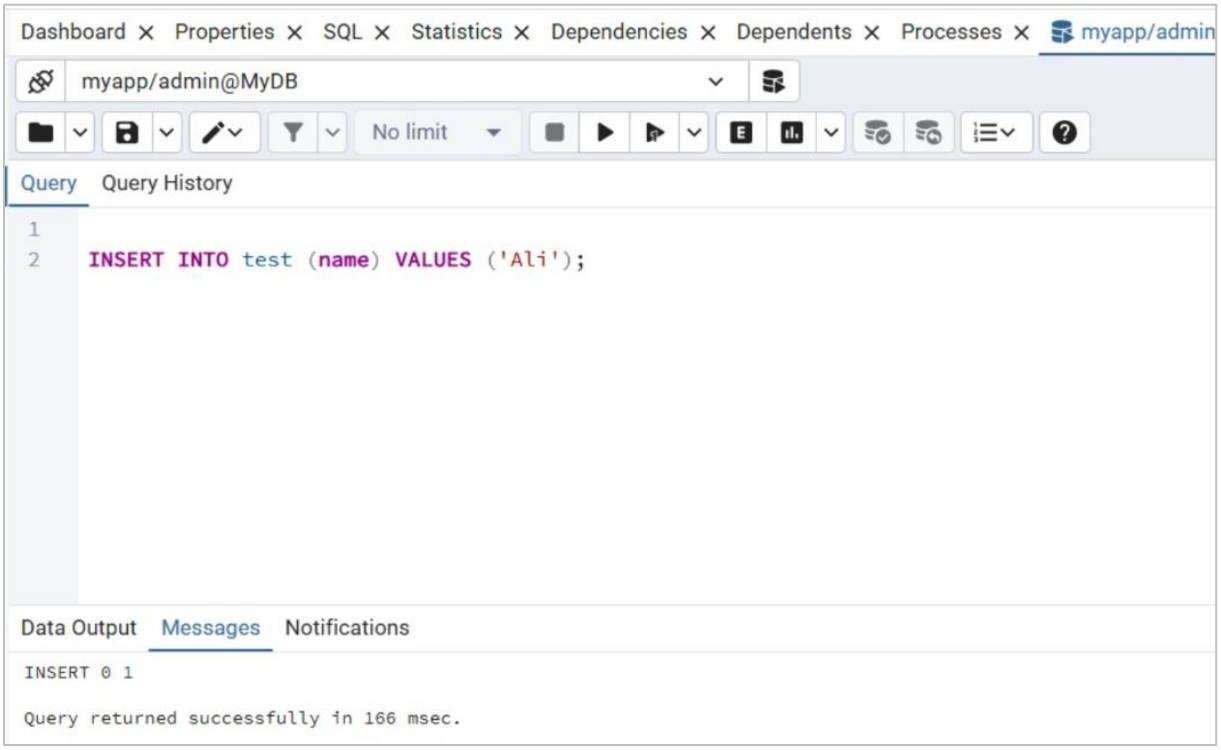

Register - Server
General
Connection
Parameters
SSH Tunnel
Advanced
Post Connection SQL
Tags
Name
MyDB
Server group
Servers
Background
Foreground
Connect now?
Shared?
Shared Username
Comments

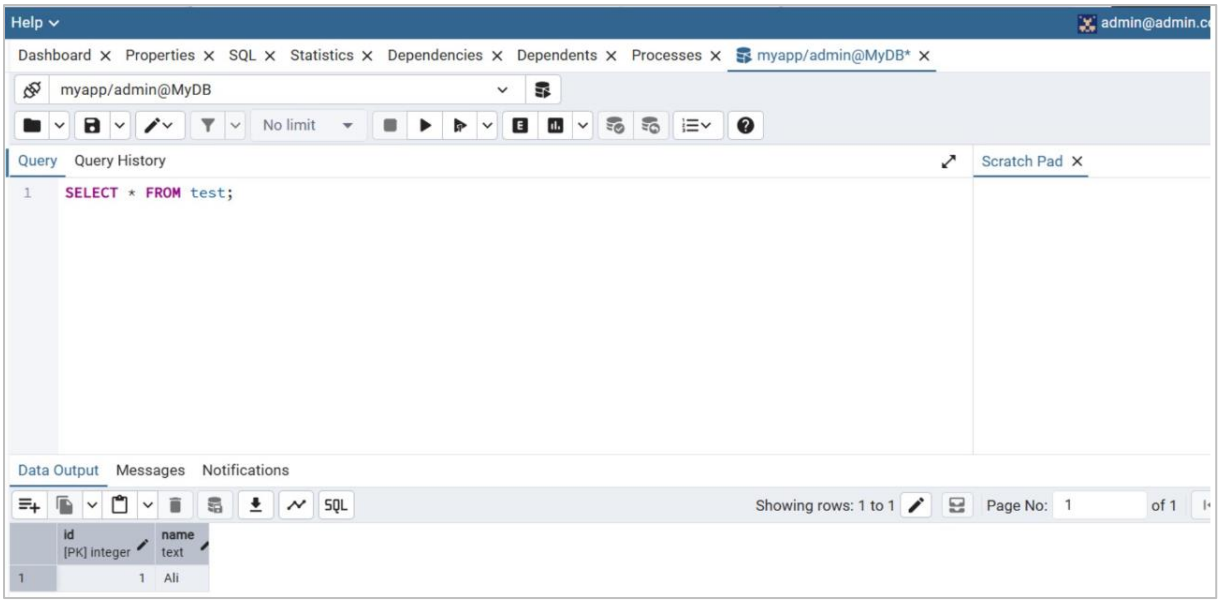




<u>Un exemple pour le test :</u>


admin
File
Object
Tools
Help
Object Explorer
Dashboard
Properties
SQL
Statistics
Dependencies
Dependents
Processes
myapp/admin@MyDB*
Servers (1)
MyDB
Databases (2)
myapp
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas
Subscriptions
postgres
Login/Group Roles
Tablespaces
myapp/admin@MyDB
No limit
Query
Query History
CREATE TABLE test (
id SERIAL PRIMARY KEY,
name TEXT
);
Data Output
Messages
Notifications
CREATE TABLE
Query returned successfully in 172 msec.


Dashboard
Properties
SQL
Statistics
Dependencies
Dependents
Processes
myapp/admin
myapp/admin@MyDB
No limit
Query
Query History
INSERT INTO test (name) VALUES ('Ali');
Data Output
Messages
Notifications
INSERT 0 1
Query returned successfully in 166 msec.



Volume persistant — vérification

```
# Arrêter les conteneurs (sans supprimer les volumes)
$ docker-compose down

# Redémarrer
$ docker-compose up

# Vérifier les volumes Docker existants
$ docker volume ls

# Supprimer les conteneurs ET les volumes (reset complet)
$ docker-compose down -v
```

Résultats

L'application multi-conteneurs fonctionne correctement :

- Le frontend est accessible
- La base de données est opérationnelle
- pgAdmin permet la gestion de la base
- Les données sont persistantes grâce au volume Docker

Conclusion

Ce TP a permis de comprendre les concepts fondamentaux de Docker, notamment : La création et gestion de conteneurs, L'orchestration avec Docker Compose,la communication entre services, la persistance des données avec les volumes, Ces compétences sont essentielles pour le développement et le déploiement d'applications modernes.